```
sendto(sockfd,buffer,strlen(buffer)+1,0,
       (struct sockaddr*)&serverAddr,sizeof(serverAddr));

recvfrom(sockfd,buffer,sizeof(buffer),0,NULL,NULL);

printf("\nTranslated Sentence:\n%s\n",buffer);

close(sockfd);

return 0;
}
```

output:
//server
UDP Server Running...
Received: omg

//client
Enter Sentence:
omg
Translated Sentence:
oh my God

Exp 3:

server:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define MAX_CLIENTS 10

int clients[MAX_CLIENTS];
int clientCount = 0;
pthread_mutex_t lock;

void broadcast(char message[], int sender)
{
   pthread_mutex_lock(&lock);
```

```
    for(int i = 0; i < clientCount; i++)
    {
        if(clients[i] != sender)
        {
            send(clients[i], message, strlen(message), 0);
        }
    }

    pthread_mutex_unlock(&lock);
}

void *handleClient(void *arg)
{
    int clientSocket = *(int *)arg;
    char buffer[1024];

    while(1)
    {
        int bytes = recv(clientSocket,buffer,sizeof(buffer), 0);
        if(bytes <= 0)
            break;
            buffer[bytes] = '\0';
            printf("Client %d: %s", clientSocket, buffer);
            broadcast(buffer, clientSocket);
    }
    close(clientSocket);

    pthread_mutex_lock(&lock);

    for(int i=0;i<clientCount;i++)
    {
        if(clients[i]==clientSocket)
        {
            for(int j=i;j<clientCount-1;j++)
            {
                clients[j]=clients[j+1];
            }

            clientCount--;
            break;
        }
    }

    pthread_mutex_unlock(&lock);
```

```
    free(arg);
    pthread_exit(NULL);
}

int main()
{
    int serverSocket;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrSize;

    pthread_mutex_init(&lock, NULL);

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if(serverSocket < 0)
    {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(5746);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    if(bind(serverSocket, (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Bind failed");
        close(serverSocket);
        exit(EXIT_FAILURE);
    }

    if(listen(serverSocket, MAX_CLIENTS) < 0)
    {
        perror("Listen failed");
        close(serverSocket);
        exit(EXIT_FAILURE);
    }

    printf("Chat Server Running...\n");

    while(1)
    {
```

```
        addrSize = sizeof(clientAddr);

        int *clientSocket = malloc(sizeof(int));

        *clientSocket = accept(serverSocket,(struct sockaddr *)&clientAddr,&addrSize);

        if(*clientSocket < 0)
        {
            perror("Accept failed");
            free(clientSocket);
            continue;
        }

        pthread_mutex_lock(&lock);

        if(clientCount < MAX_CLIENTS)
        {
            clients[clientCount++] = *clientSocket;
        }
        else
        {
            printf("Maximum clients reached.\n");
            close(*clientSocket);
            free(clientSocket);
            pthread_mutex_unlock(&lock);
            continue;
        }

        pthread_mutex_unlock(&lock);

        pthread_t tid;

        pthread_create(&tid, NULL, handleClient, clientSocket);
        pthread_detach(tid);
    }

    close(serverSocket);
    pthread_mutex_destroy(&lock);

    return 0;
}


client:
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>

#define PORT 4213
#define BUFFER_SIZE 1024

int sockfd;

void *receiveMessage(void *arg)
{
    char buffer[BUFFER_SIZE];

    while(1)
    {
        int bytes = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);

        if(bytes <= 0)
        {
            printf("\nDisconnected from server.\n");
            break;
        }

        buffer[bytes] = '\0';

        printf("\nMessage: %s", buffer);
    }

    pthread_exit(NULL);
}

int main()
{
    struct sockaddr_in serverAddr;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if(sockfd < 0)
    {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
```

```
    }

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    if(connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Connection failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("Connected to Chat Server.\n");

    pthread_t tid;
    pthread_create(&tid, NULL, receiveMessage, NULL);

    char buffer[BUFFER_SIZE];

    while(1)
    {
        if(fgets(buffer, sizeof(buffer), stdin) == NULL)
            break;

        send(sockfd, buffer, strlen(buffer), 0);
    }

    close(sockfd);
    return 0;
}

output:
 //server
Chat Server Running...

//client
Connected to Chat Server.
hii
Message: hiiii
omg
Message: helloo
```

Exp 4:

server:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<arpa/inet.h>
#include<unistd.h>
#include<time.h>

int main()
{
        int server_sock;
        char buffer[1024],now[100];
        time_t t;
        struct sockaddr_in serverAddr, clientAddr;
        server_sock = socket(AF_INET, SOCK_DGRAM, 0);
        serverAddr.sin_family = AF_INET;
        serverAddr.sin_port = htons(7564);
        serverAddr.sin_addr.s_addr = INADDR_ANY;
        bind(server_sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr));
        printf("UDP Server Running...\n");
        socklen_t len=sizeof(clientAddr);
        while(1)
        {
                recvfrom(server_sock, buffer, sizeof(buffer), 0,(struct sockaddr
*)&clientAddr,&len);
        time(&t);
        strcpy(now,ctime(&t));
        sendto(server_sock,now,sizeof(now),0,(struct sockaddr*)&clientAddr,len);
      }
        close(server_sock);
        return 0;
}
```

client:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
```